Sprawozdanie z projektu 1

Temat projektu:

Gra "Saper"w języku C++

Wykonał:

Filip Dawid Nr albumu: 193204

Maja Czarniecka Nr albumu: 193327

Przedmiot:

Organizacja systemów komputerowych - laboratorium

Gdańsk, 18 marca 2026

Spis treści

1 Wstęp i sformułowanie zadania **2**
1.1 Cel projektu 2
1.2 Funkcjonalności programu 2
1.3 Wybór środowiska i technologii 2

2 Architektura danych i kluczowe algorytmy **3**
2.1 Reprezentacja planszy w pamięci komputera 3
2.2 Zabezpieczenie pierwszego ruchu 3
2.3 Algorytm obliczania wartości pól (Sąsiadujące miny) 4
2.4 Algorytm Flood Fill (Odkrywanie pustych przestrzeni) 4

3 Zarządzanie stanem oraz interfejs graficzny **5**
3.1 Główna pętla gry i maszyna stanów (State Machine) 5
3.2 Obsługa zdarzeń asynchronicznych (Event Polling) 5
3.3 Autorski system interfejsu (GUI) 5

4 Zaawansowane mechaniki gry **6**
4.1 Mechanika Undo (Cofanie czasu) 6
4.2 System zamrażania czasu (Pauza) 7
4.3 Mechanika Chording (Szybkie odsłanianie) 7
4.4 Trzystanowy system oznaczania 8
4.5 Zarządzanie wejściem/wyjściem i Tabela Wyników 8

5 Podsumowanie i wnioski **8**
5.1 Ocena realizacji założeń 8
5.2 Specyfika środowiska i dystrybucja pliku wykonywalnego 9
5.3 Zalety zaimplementowanej aplikacji 9
5.4 Napotkane problemy i debugging 9
5.5 Kierunki dalszego rozwoju i wady architektury 9

1 Wstęp i sformułowanie zadania

1.1 Cel projektu

Głównym celem zrealizowanego projektu było zaimplementowanie w pełni funkcjonalnej gry logicznej „Saper” (ang. Minesweeper). Aplikacja została napisana w języku C++ i wyposażona w graficzny interfejs użytkownika (GUI). W ramach założeń projektowych gra miała wykorzystywać technikę tworzenia graficznego interfejsu użytkownika, który powinien w sposób interaktywny wykorzystywać do komunikacji z operatorem monitor ekranowy, klawiaturę oraz myszkę.

1.2 Funkcjonalności programu

- **Tryby rozgrywki:** Zaimplementowanie standardowych rozmiarów plansz: Początkujący (8×8, 10 min), Zaawansowany (16×16, 40 min), Ekspert (30×16, 99 min) oraz trybu „Plansza Użytkownika”, w którym gracz może samemu określić wymiary siatki oraz ilość min.

- **Mechanika „First Click Safety”:** Zabezpieczenie pierwszej tury gracza. Inicjalizacja ładunków wybuchowych na planszy musi nastąpić dopiero po pierwszym kliknięciu, co gwarantuje matematyczną pewność, że pierwsze odkryte pole zawsze będzie puste.

- **Algorytm odsłaniania mapy:** Automatyczne zwalnianie rozległych, pustych przestrzeni przy użyciu algorytmu przeszukiwania grafu w celu minimalizacji liczby wymaganych kliknięć ze strony użytkownika.

- **Opcje dodatkowe:** Wdrożenie opcjonalnych mechanik, takich jak *Chording* (szybkie odkrywanie sąsiadów weryfikujące poprawność oflagowania), możliwość postawienia znaku zapytania, system podpowiedzi (Hint) oraz cofanie ruchów (Undo) po przypadkowej detonacji.

- **Zarządzanie stanem i UI:** Stworzenie lokalnej, zapisywanej na dysku tabeli wyników (Leaderboard), implementacja pauzy zamrażającej stoper gry oraz możliwość zmiany motywów graficznych (skórek).

1.3 Wybór środowiska i technologii

Do realizacji warstwy wizualnej wybrano bibliotekę **SFML** w wersji 2.6. Wybór ten podyktowany był jej wysoką wydajnością, wsparciem dla architektury C++ podczas pracy na systemach Windows i Linux, prostotą obsługi wejścia/wyjścia (mysz, klawiatura). Projekt skompilowano z wykorzystaniem kompilatora G++.

2 Architektura danych i kluczowe algorytmy

2.1 Reprezentacja planszy w pamięci komputera

W projekcie zdecydowano się na podejście do implementacji plansz dwuwymiarowych zoptymalizowane pod kątem wykorzystania pamięci podręcznej procesora - użyto **wektora jednowymiarowego**.

Tablice wielowymiarowe często cierpią z powodu fragmentacji pamięci (wiersze siatki mogą znajdować się w odległych sektorach pamięci RAM). Użycie płaskiego wektora `std::vector<Cell> grid;` gwarantuje, że cała plansza leży w ciągłym bloku pamięci. Aby uzyskać dostęp do konkretnej komórki posiadając współrzędne (X, Y), zastosowano funkcję transformującą:

$$Index = Y \cdot szerokosc_planszy + X$$

```
struct Cell {
    bool isMine = false;
    bool isRevealed = false;
    int flagState = 0;          // 0: brak, 1: flaga, 2: znak zapytania
    int adjacentMines = 0;
};

inline int getIndex(int x, int y) {
    return y * columns + x;
}
```

Listing 1: Struktura komórki i funkcja mapująca współrzędne

2.2 Zabezpieczenie pierwszego ruchu

Zaimplementowany algorytm losujący pozycję min wykorzystuje generator liczb pseudolosowych z nagłówka `<random>`. Wywoływany jest on dopiero podczas przechwycenia pierwszego lewego kliknięcia myszą w stanie `Playing`. Funkcja generująca przyjmuje jako argumenty współrzędne (x, y) kliknięgo pola.

Podczas losowania w pętli `while (minesPlaced < minesCount)`, algorytm weryfikuje wylosowane indeksy. Jeżeli wygenerowana pozycja pokrywa się ze współrzędnymi (x, y) przekazanymi w argumencie (lub miny zostały tam już postawione w poprzedniej iteracji), przypisanie ładunku jest przerywane i pętla wykonuje kolejną iterację, aż do momentu bezpiecznego rozmieszczenia wymaganej puli min. Ostatnim krokiem jest jednorazowe przeliczenie zmiennej `adjacentMines` dla każdej komórki planszy.

2.3 Algorytm obliczania wartości pól (Sąsiadujące miny)

Ostatnim krokiem inicjalizacji planszy, następującym bezpośrednio po wylosowaniu min, jest jednorazowe przeliczenie zmiennej `adjacentMines` dla każdej komórki. Algorytm iteruje przez całą siatkę, a dla każdego pola niebędącego miną, sprawdza 8 sąsiednich pozycji. Jeżeli sąsiadujące pole kryje ładunek wybuchowy, licznik jest inkrementowany. Kluczowym aspektem jest weryfikacja granic planszy za pomocą funkcji `isValid()`, co zapobiega odwołaniom do niezaalokowanej pamięci.

```
for (int y = 0; y < rows; ++y) {
    for (int x = 0; x < columns; ++x) {
        if (!grid[getIndex(x, y)].isMine) {
            int mineCount = 0;
            // Przeszukanie 8 sasiadow (zagniezdzone petle -1 do 1)
            for (int dy = -1; dy <= 1; ++dy) {
                for (int dx = -1; dx <= 1; ++dx) {
                    if (isValid(x + dx, y + dy) && grid[getIndex(x + dx,
    y + dy)].isMine) {
                        mineCount++;
                    }
                }
            }
            grid[getIndex(x, y)].adjacentMines = mineCount;
        }
    }
}
```

Listing 2: Obliczanie liczby sasiadujacych min dla kazdego pola

2.4 Algorytm Flood Fill (Odkrywanie pustych przestrzeni)

Jednym z najważniejszych aspektów gry w Sapera jest automatyczne odsłanianie bezpiecznych stref, gdy gracz trafi na pole z wartością "0"(brak min w sąsiedztwie). Realizowane jest to za pomocą rekurencyjnego wariantu algorytmu **Flood Fill**.

Gdy funkcja `revealCell(x, y)` napotyka pole o wartości `adjacentMines == 0`, wywołuje samą siebie dla wszystkich 8 sąsiadujących kierunków (N, S, W, E oraz przekątnych). Aby zapobiec nieskończonej rekurencji oraz błędom pamięci, przed wywołaniem następuje weryfikacja funkcji pomocniczej `isValid(nx, ny)`, która upewnia się, że indeksy nie wykraczają poza fizyczne granice wektora.

3 Zarządzanie stanem oraz interfejs graficzny

3.1 Główna pętla gry i maszyna stanów (State Machine)

Architektura gier z użyciem SFML wymusza stosowanie stałej pętli (`while (window.isOpen())`). W celu uniknięcia głębokiego zagnieżdżania instrukcji warunkowych `if-else`, strukturę aplikacji oparto na abstrakcji Maszyny Stanów (ang. State Machine). Wykorzystano mocno typowany typ wyliczeniowy `enum class`:

```
enum class GameState {
    Menu, Playing, GameOver, Victory, EnterName, Leaderboard, Paused
};
extern GameState currentState;
```

Listing 3: Definicja maszyny stanów

Procedury takie jak `updateUI()` (obliczająca logikę, pozycję kursora) oraz `renderUI()` (wyświetlanie GUI) wykorzystują `currentState` jako selektor strumienia logiki. Dzięki temu zdarzenia z klawiatury w stanie `Menu` nie powodują efektów ubocznych w stanie `Playing`.

3.2 Obsługa zdarzeń asynchronicznych (Event Polling)

Komunikacja aplikacji z systemem operacyjnym jest realizowana poprzez kolejkę zdarzeń biblioteki SFML, odczytywaną za pomocą funkcji `window.pollEvent(event)`. Zdarzenia takie jak: wciśnięcie klawisza, kliknięcie przycisku myszy czy zamknięcie okna są przetwarzane sekwencyjnie przed wykonaniem logiki gry w danej klatce. Zaimplementowano skomplikowaną obsługę wejścia znakowego (`sf::Event::TextEntered`), pozwalającą użytkownikowi na natywne wpisywanie tekstu (np. swojego imienia po wygranej lub własnych parametrów planszy), uwzględniając usuwanie znaków klawiszem *Backspace*.

3.3 Autorski system interfejsu (GUI)

Aby nie obciążać projektu instalacją zewnętrznych nakładek GUI, zaprogramowano od podstaw własny, proceduralny system generowania przycisków retro. Każdy interaktywny element menu opiera się na wywołaniu autorskiej funkcji `drawRetroButton`.

Funkcja przyjmuje referencję do okna renderującego oraz obiektu typu `sf::Text`. Przy pomocy metody `getGlobalBounds()` generuje obrys przestrzenny przycisku. Sprawdzenie, czy metoda `bounds.contains(mousePos)` zwraca prawdę, decyduje o aktywacji efektu najechania (zmiana barwy tła, przesunięcie geometryczne imitujące wciśnięcie). Dzięki takiemu podejściu, dodawanie nowych rozwijanych list, czy skomplikowanych pasków menu w trakcie trwania rozgrywki, sprowadziło się jedynie do iteracyjnego wywoływania gotowych bloków wewnątrz pętli.

Ponadto autor przygotował własne ikony pól gry postaci plików .png o rozmiarze 32x32 w trzech różnych motywach w stylu retro pixelart:

- Classic - wzorowane na klasycznym wyglądzie Sapera
- Modern - nowoczesny niebieski motyw
- Green - zielony motyw z kontrastującymi flagami

Z technicznego punktu widzenia, pliki graficzne są ładowane do pamięci VRAM jako obiekty `sf::Texture`. Następnie do ich sprawnego rysowania na ekranie wykorzystano klasę `sf::Sprite`. Pozwala to w czasie rzeczywistym przypisywać wskaźniki odpowiednich tekstur (np. mina, flaga, puste pole) do renderowanych w pętli *sprite'ów* reprezentujących komórki planszy.

4 Zaawansowane mechaniki gry

Podstawowa wersja gry Saper jest zbiorem dość prostych i przewidywalnych reguł: po kliknięciu danego pola ukazuje się liczba sąsiadujących min, które należy znaleźć; gra kończy się po oflagowaniu wszystkich min. Aby zwiększyć dostępność dla gracza zaimplementowano dodatkowe mechaniki, znacząco rozszerzające standardową pętlę rozgrywki.

4.1 Mechanika Undo (Cofanie czasu)

Przed każdym kliknięciem lewym przyciskiem myszy, które dotyczy pola zakrytego, wywoływana jest funkcja `saveState()`. Funkcja ta dokonuje pełnej, głębokiej kopii stanu gry:

```
void saveState() {
    previousGrid = grid;  // Kopia zapasowa wektora
    prevFlagsPlaced = flagsPlaced;
}

void undoState() {
    grid = previousGrid;  // Nadpisanie glownego wektora
    flagsPlaced = prevFlagsPlaced;
    currentState = GameState::Playing; // Ozywienie gry po wybuchu
}
```

Listing 4: Implementacja zapisu i przywracania stanu wektora

Taka operacja, ze względu na płaską strukturę jednowymiarowego wektora, wykonuje się niezwykle szybko i ma znikomy wpływ na zasoby, oferując graczowi bezcenne w trudnych momentach narzędzie ułatwiające rozgrywkę.

4.2 System zamrażania czasu (Pauza)

Zegar gry (`sf::Clock`) mierzy upływ od rozpoczęcia rozgrywki bez przerwy. Aby uzyskać prawdziwy efekt zatrzymania wymiaru czasowego, wdrożono zmienną buforową `savedTime`. Moment wciśnięcia klawisza **P** sprawia, że aktualny zliczony czas (wyrażony w sekundach) trafia do bufora. Podczas wznowienia rozgrywki, stoper biblioteki SFML jest restartowany, a do odmierzanego czasu na bieżąco dodawana jest zbuforowana wartość.

```
if (event.key.code == sf::Keyboard::P) {
    if (currentState == GameState::Playing && !isFirstClick) {
        currentState = GameState::Paused;
        savedTime = elapsedTime; // Zrzut obecnego czasu do bufora
    } else if (currentState == GameState::Paused) {
        currentState = GameState::Playing;
        gameClock.restart(); // Restart zegara sprzetowego
    }
}
// Sumowanie czasu w petli glownej:
// elapsedTime = savedTime + gameClock.getElapsedTime().asSeconds();
```

Listing 5: Mechanika pauzy oparta na buforze czasu

4.3 Mechanika Chording (Szybkie odsłanianie)

Dla zaawansowanych graczy zaimplementowano mechanikę zwaną *Chordingiem*. Jeżeli gracz odkrył już pole z cyfrą i prawidłowo oflagował wokół niego odpowiednią liczbę min, ponowne kliknięcie na ową cyfrę wyzwala algorytm. Oblicza on sumę sąsiednich flag, a w przypadku zgodności z cyfrą – automatycznie odkrywa pozostałe nieoflagowane pola.

```
int flagCount = 0;
for (int dy = -1; dy <= 1; ++dy) {
    for (int dx = -1; dx <= 1; ++dx) {
        if (isValid(x + dx, y + dy)) {
            if (grid[getIndex(x + dx, y + dy)].flagState == 1) flagCount
    ++;
        }
    }
}
// Jezeli ilosc flag zgadza sie z liczba na polu, odkrywamy reszte
    sasiadow
if (flagCount == grid[getIndex(x, y)].adjacentMines) {
    // Petla odkrywajaca sasiadow...
}
```

Listing 6: Algorytm sprawdzajacy liczbe flag wokol pola

4.4 Trzystanowy system oznaczania

Standardowy system flagowania (prawy przycisk myszy) został rozszerzony o trzeci stan – znak zapytania (?). Cykliczne przełączanie stanów zapobiega omyłkowemu wyzwoleniu mechaniki chordingu dla niepewnych pól.

```
if (grid[idx].flagState == 0) {
    grid[idx].flagState = 1; // Ustaw flage
    flagsPlaced++;
} else if (grid[idx].flagState == 1) {
    flagsPlaced--;
    grid[idx].flagState = optQuestionMarks ? 2 : 0; // Ustaw pytajnik
   lub wyczysc
} else if (grid[idx].flagState == 2) {
    grid[idx].flagState = 0; // Wyczysc
}
```

Listing 7: Logika obslugi stanow pod prawym przyciskiem myszy

4.5 Zarządzanie wejściem/wyjściem i Tabela Wyników

Osiągnięcia graczy są archiwizowane w sposób trwały przy wykorzystaniu standardowych operacji na strumieniach `<fstream>`. Struktura danych grupuje wyniki według poziomów trudności. Zapis odbytej rozgrywki wymusza na programie natychmiastowe nadpisanie struktury pliku tekstowego `leaderboard.txt`.

```
void saveLeaderboard() {
    std::ofstream file("assets/leaderboard.txt");
    if (file.is_open()) {
        for (int i = 0; i < 4; ++i) { // 4 poziomy trudnosci
            for (const auto& entry : leaderboards[i]) {
                file << i << " " << entry.name << " " << entry.time << "
   \n";
            }
        }
        file.close();
    }
}
```

Listing 8: Funkcja archiwizujaca tabele wynikow na dysku

5 Podsumowanie i wnioski

5.1 Ocena realizacji założeń

Zastosowanie wydajnego języka C++ i silnika SFML zaowocowało błyskawicznym czasem reakcji gry - niezależnie od tego, czy gracz operuje na planszy z 64 czy z 1000 pól, alokacja,

renderowanie oraz rekurencyjne odkrywanie siatki następuje poniżej pojedynczej klatki renderowania (< 16 milisekund przy zadanym progu 60 FPS).

5.2 Specyfika środowiska i dystrybucja pliku wykonywalnego

Aplikacja była projektowana z myślą o natywnym uruchamianiu w systemach rodziny Windows. Zastosowano flagę `-mwindows`. Dzięki niej finalny plik `saper.exe` uruchamia się jako aplikacja graficzna Win32 bez otwierania klasycznego, czarnego okna konsoli poleceń w tle. Do poprawnej dystrybucji na inne jednostki, niezbędne było scalenie pliku wykonywalnego z powiązanymi dynamicznymi bibliotekami C++ (plikami DLL) takimi jak `libstdc++-6.dll` oraz bibliotekami systemu SFML. Dodatkowo zintegrowano plik zasobów `icon.rc`, pozwalający na nadanie aplikacji własnej, natywnej ikony Windows.

5.3 Zalety zaimplementowanej aplikacji

W porównaniu z tradycyjnym Saperem wbudowanym w starsze wersje systemu Windows, nowa aplikacja oferuje niespotykane, zaawansowane narzędzia wsparcia gracza. Systemy takie jak „Pierwszy bezpieczny ruch" diametralnie zmniejszają poziom frustracji losowością gry, a cofanie ruchów zachęca użytkowników do podejmowania ryzyka logicznego. Utrzymanie wszystkiego w estetyce pixel-art wraz z interfejsem graficznym znacznie poprawia odbiór wizualny.

5.4 Napotkane problemy i debugging

W trakcie prac programistycznych głównym problemem była **responsywność na najmniejszej rozdzielczości:** W trybie Początkującym rozmiar okna wynosił zaledwie 8 kolumn szerokości. Po dodaniu funkcjonalności (opcje, pauza, timer), przyciski interfejsu najeżdżały na siebie, powodując błędne wyliczanie zdarzeń `Hover`. Rozwiązaniem było wprowadzenie minimalnej szerokości okna `MIN_WINDOW_WIDTH = 550`, które pozwala wyśrodkować planszę względem osi X i zachować marginesy interfejsu.

5.5 Kierunki dalszego rozwoju i wady architektury

Mimo wysokiej stabilności działania w projekcie rozproszono logikę pomiędzy plikami w oparciu o globalne zmienne (`extern`), zmniejsza skalowalność kodu w przypadku dodawania nowych elementów oprogramowania.

Zdefiniowanie klasy do której jako prywatne właściwości trafiłyby wektor siatki, wymiary oraz metody odkrywania, oddzielałoby zasoby gry.